Utilisation d’un stockage S3 dans une application PHP

1. Introduction aux principes d’un stockage S3 . . . 1
1.1. L’API S3 . . . 2
1.2. Organisation d’un stockage s3 : buckets et objets . . . 2
1.3. Bonnes pratiques concernant les clés . . . 3
1.4. Bucket privés et publics . . . 4
2. Mise en place de l’environnement . . . 5
3. Programmation de l’api . . . 8
3.1. Création du client s3 . . . 8
3.2. Le service de gestion du stockage (StorageService) . . . 9
3.3. Programmation de l’api Rest . . . 14
4. Administration du stockage S3, CLI . . . 15
4.1. Utilisation de l’interface d’administration de SeaweedFS . . . 16
4.2. Utilisation du client cli aws s3 . . . 16

Introduction et contexte

L’objectif de ce tutoriel est de comprendre le fonctionnement et l’utilisation d’un stockage d’objets de type Amazon S3 au travers de sa mise en oeuvre pas à pas au sein d’une application écrite en PHP. Le cas d’utilisation illustré dans la suite est celui du stockage d’images et on traitera deux fonctionnalités :

- l’upload de fichiers via un POST
- la génération et l’utilisation d’URL d’accès direct aux images

L’intégralité du code servant à illustrer ce tutoriel est disponible ici : https://gitlab.univ-lorraine.fr/canals5/s3_php.git

1. Introduction aux principes d’un stockage S3

S3 est le service de stockage de données conçu et développé par Amazon au sein de Amazon Web Services (AWS). Amazon propose de nombreux outils pour manipuler ce stockage, notamment des SDK pour différents langages ainsi que des outils de commande en ligne.

L’utilisation de S3 s’est généralisée, et il existe maintenant de nombreux outils et services *compatibles S3* qui offrent des services identiques sans avoir à utiliser AWS. On peut par exemple citer Garage, Ceph, MinIO, RustFS ou **SeaweedFS**. Ces outils permettent ainsi d’auto-héberger un service de stockage S3, tout en conservant une compatibilité AWS totale :

- ces outils et services sont interchangeables, y compris avec AWS
- les SDK AWS sont parfaitement utilisables avec tous les stockage compatibles s3, ce sont donc ces sdk que l'on utilise,
- de même, les outils AWS cli sont utilisables avec ces stockages compatibles.

Dans la suite, nous utiliserons l'outil open-source SeaweedFS (https://github.com/seaweedfs/seaweedfs), au sein d'une composition Docker.

1.1. L'API S3

Toutes les opérations ne manipulation de données au sein d'un stockage S3 se font au travers de l'API S3. L'API S3 est une API REST HTTP : toutes les opérations se font via des requêtes HTTP standards (GET, PUT, DELETE…). C'est ce qui fait sa force — n'importe quel langage ou outil capable de faire du HTTP peut interagir avec S3. SeaweedFS, ou les autres stockage compatibles S3, implémente cette même API, donc tous les outils et applications utilisant cette api peuvent fonctionner avec SeaweedFS ou un autre stockage S3.

L'interaction avec cette api peut se faire de différentes manières :

1. en ligne de commande en utilisant l'outil `aws s3` : pratique pour des opérations d'administration du stockage, ponctuelles ou scriptées,
2. au sein d'une application, en utilisant un SDK fourni pas AWS. Il existe des sdk officiels pour la plupart des langages,
3. en utilisant des URL directes, permettant un accès direct à un objet stocké, pratique par exemple pour des assets publics web (images, css…)

1.2. Organisation d'un stockage s3 : buckets et objets

1.2.1. la notion de bucket

Un stockage S3 est structuré autour de la notion de *bucket*. Un bucket est le conteneur de base dans S3. C'est l'unité d'organisation principale dans laquelle on stocke des données. Pour faire une analogie simple :

- Un bucket ≈ un disque dur ou un répertoire racine
- Les objets qu'il contient ≈ les fichiers sur ce disque

Quelques caractéristiques importantes :

- Le nom d'un bucket doit être unique au sein d'une instance S3
- Un bucket n'a pas de limite de taille théorique
- On peut créer plusieurs buckets pour organiser ses données (par projet, par type, par environnement, etc.)
- Un bucket peut contenir une quantité illimitée d'objets

1.2.2. les objets

Un *objet* est l'unité de stockage dans S3. Chaque objet est composé de trois éléments :

- La clé (key) : identifiant unique de l'objet dans le bucket, qui permet de désigner et accéder à l'objet,
- le contenu (body) : les données sous forme binaire brute,
- Les métadonnées : Des informations associées à l'objet (`Content-Type`, date de création, tags personnalisés…)

S3 est un stockage orienté objet généraliste : vous pouvez y stocker n'importe quel type de données. Par exemple :

- Images, vidéos, audio : .jpg, .mp4, .mp3
- Documents : .pdf, .docx, .csv, .txt, .opd
- Archives et backups : .tar.gz, .zip, .bak
- Données applicatives : fichiers JSON, XML, logs
- Assets web : HTML, CSS, JS, fonts
- Binaires : executables, images Docker (layers), fichiers compilés
- …

La taille maximale d'un objet est généralement de 5 To (avec le mécanisme de multipart upload pour les gros fichiers).

1.2.3. structure d'un bucket, organisation des clés

Un bucket est un **stockage plat**, organisé comme une table `clé → objet`.

Cependant, il est d'usage de structurer les valeurs des clés sous la forme de chemin de fichier en incluant des `/`. Ceci permet de simuler une hiérarchie.

```
mon-bucket/
├─── images/
│    ├─── a45GT5642!Erf29.png         → clé : "images/a45GT5642!Erf29.png"
│    └─── re5498HAXF3.jpg             → clé : "images/re5498HAXF3.jpg"
└─── backups/
     └─── 2026/data.tar.gz            → clé : "backups/2026/data.tar.gz"
```

Pour autant, ce sont bien des objets à plat dans le bucket, mais les outils S3 (comme `aws s3 ls`) affichent ça comme une arborescence pour plus de clarté.

1.3. Bonnes pratiques concernant les clés

Le choix de l'organisation et des valeurs des clés est important car le nombre d'objets au sein d'un bucket peut être très grand. De mauvais choix peuvent conduire à des situations complexes à gérer.

Deux principes importants à appliquer de manière systématique :

- les clés doivent être uniques au sein d'un bucket ; compte tenu du nombre potentiellement illimité d'objets dans un bucket, il faut choisir des valeurs de clés permettant d'éviter les collisions même dans le cas d'un grand nombre d'objets,
- il faut utiliser systématiquement la hiérarchisation virtuelle des clés, de manière à trier les objets (par type, par usage, par application ...) et afin de limiter les collisions.

1.3.1. Cas des clés pour des fichiers uploadés par les utilisateurs

Lorsque l'application prévoit la possibilité pour ses utilisateurs d'uploader des fichiers (documents, images, vidéos ...), quelques précautions sont à prendre.

1. **Ne jamais utiliser le nom de fichier original** : il est rarement unique et donc sujet à collisions fréquentes. De plus un nom devinable peut poser des problèmes de sécurité ou de confidentialité.
2. **Utiliser systématiquement un UUID** dans une partie de la clé, pour éviter les collisions et les problèmes de sécurité.

   `images/3f2a1b4c-5d6e-7f8a-9b0c-1d2e3f4a5b6c.jpg`

3. **Conserver l'extension du fichier** (.pdf, .jpg ...) qui sera utile pour le `Content-Type`, et facilitera ainsi l'utilisation via une URL directe ; le nom original peut être stocké en méta-données ou en BD.

   ```
   # Clé S3
   images/3f2a1b4c-5d6e-7f8a-9b0c-1d2e3f4a5b6c.jpg

   # En base de données
   { s3_key: "images/3f2a1b4c-5d6e-7f8a-9b0c-1d2e3f4a5b6c.jpg",
     original_name: "vacances_été.jpg",
     user_id: 42
   }
   ```

4. **Organiser par utilisateur (ou autre concept métier) et si c'est utile par date**

   `images/projet/42/2026/03/{uuid}.jpg`

1.4. Bucket privés et publics

Par défaut, tout bucket S3 est privé : seule les personnes possédant les droits d'accès en lecture ou en écriture peuvent accéder au contenu du bucket. Il est bien sûr possible de gérer ces droits d'accès et de les transmettre.

Dans certains cas, il est nécessaire de de donner des accès publics à certains objets d'un bucket. Par exemple dans le cas d'asset web, qui doivent être accessibles via une URL.

Pour donner des accès publics à des objets stockés dans un bucket privé, il est possible de

demander au stockage la génération d'URLs directes dites *pré-signées* ; une URL pré-signée est une URL temporaire (valable pour une durée déterminée) permettant d'accéder à un objet précis et pour une opération déterminée (GET, PUT…). L'URL change à chaque fois qu'elle est générée.

Il est également possible de rendre des buckets **publics** en lecture où *tous* les objets sont accessibles directement par URL. Cette approche est acceptable pour un bucket stockant uniquement des données publiques non sensibles : assets web, images publiques, documents publics.

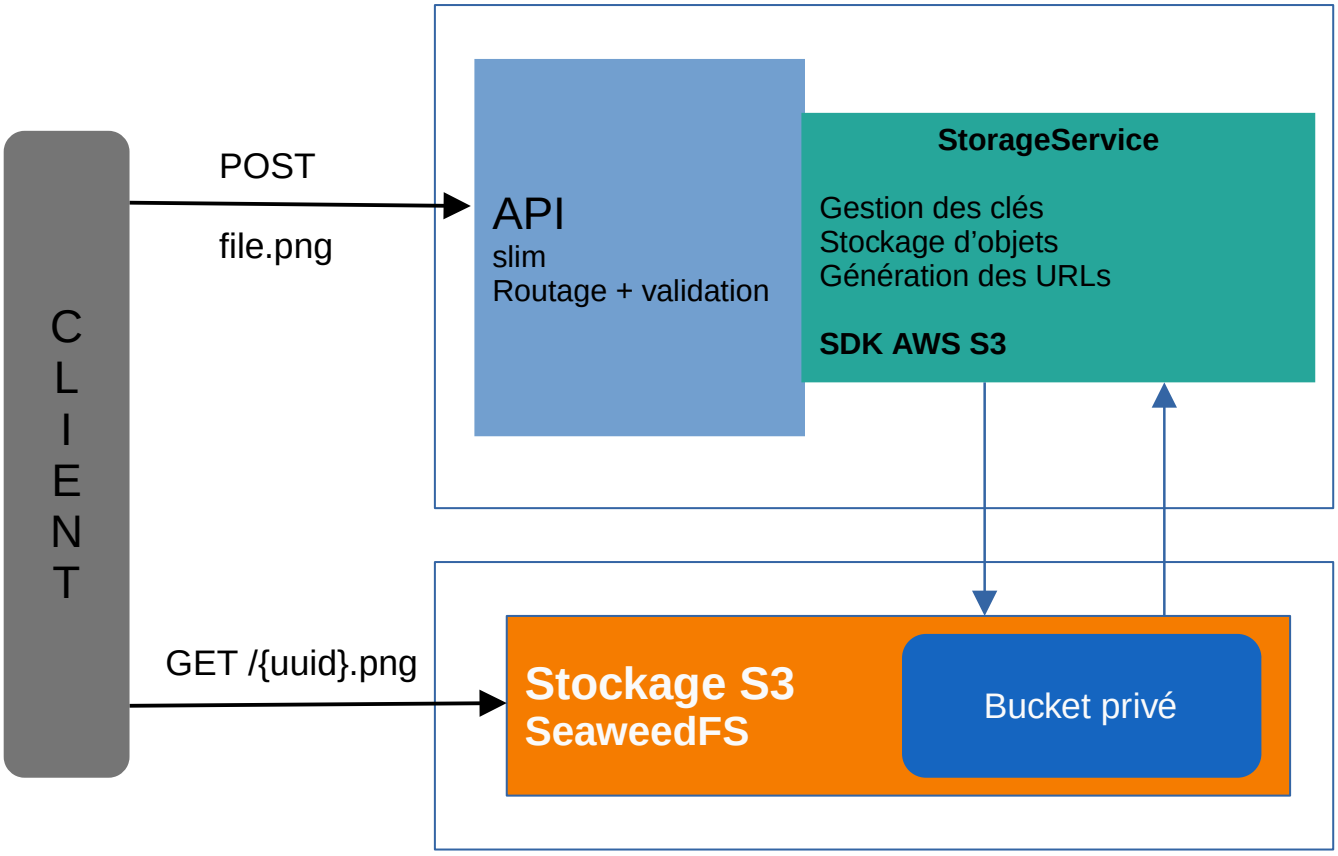
De manière générale, il faut privilégier les buckets privés et utiliser des URLs pré-signées, et réserver les buckets publics pour les cas où cela est strictement nécessaire, par exemple les cas où on doit pouvoir utiliser une URL d'accès statique.

2. Mise en place de l'environnement

Le cas d'étude est celui du stockage d'images uploadées par des utilisateurs au sein d'une application web en PHP. Ces images doivent être accessibles via une URL directe pour être visualisées au sein d'une page web.

On a donc les composants suivants dans notre application :

- une API construite avec Slim, implantant la route `POST` pour l'upload d'une image,
- une action pour traiter cet upload : elle implante le pipe-line de validation du fichier (type, taille) puis fait appel à un `StorageService` pour générer la clé, stocker le fichier et obtenir une URL pré-signée,
- un `StorageService`: le service métier implanté à l'aide du SDK AWS PHP pour interagir avec le stockage : stockage d'objet, accès, suppression d'objets…
- un service de stockage S3 réalisé avec SeaweedFS.



L'ensemble des composants (api + stockage S3) est déployé au sein d'une composition Docker. On y déclare 2 services : le service s3/SeaweedFS et un service php classique pour déployer l'api. On a donc ce docker-compose.yml :

```yaml
networks:
  S3.net:
    driver: bridge

services:
# service api : point d'entrée de l'application
  api:
    build:
      context: build
      dockerfile: 8.4-cli.Dockerfile
    ports:
      - "8080:80"
    volumes:
      - ./storage:/var/php
    working_dir: /var/php
    networks:
      - S3.net
    depends_on:
      - S3
    command: sh -c "composer install && php -S 0.0.0.0:80 -t /var/php/public"

# le service de stockage s3 avec SeaweedFS
  S3:
    image: chrislusf/seaweedfs:latest
```

```
    ports:
      - "8333:8333"
      - "9333:9333"
      - "23646:23646"
    command: "server -dir=/data -s3 -s3.port=8333 -s3.config=/etc/seaweedfs/s3.json"
    networks:
      - S3.net
    volumes:
      - ./S3/data:/data
      - ./S3/s3.json:/etc/seaweedfs/s3.json
```

En ce qui concerne le service S3 :

- l'api s3 est associée au port `8333`, indiqué dans la commande de lancement du serveur SeaweedFS : `-s3.port=8333`
- ce port est lié au port `8333` de l'hôte,
- le port `9333` permet d'accéder à la console d'état seaweedfs,
- le port `23646` permet d'accéder à l'interface d'administration du serveur (attention, le service doit être explicitement lancé en plus du serveur)
- SeaweedFS stocke ses données dans le répertoire `/data` bind mounted sur la machine hôte sur `./S3/data`
- SeaweedFS utilise un fichier de configuration `/etc/seaweedfs/s3.json` bind mounted sur la machine hôte sur `./S3/s3.json`

Ce fichier de configuration json contient essentiellement les identités, les credentials et les droits des utilisateurs. Les crédentials sont sous la forme AWS : une `accessKey` qui sert d'identifiant et une `secretKey`, équivalent d'un password.

Par exemple, deux utilisateurs, dont un bénéficiant des droits Admin :

```
{
  "identities": [
    {
      "name": "admin",
      "actions": ["Read", "Write","List", "Admin"],
      "credentials": [
        {
          "accessKey": "ABCDEF",
          "secretKey": "123456"
        }
      ]
    },
    {
      "name": "marcel",
      "actions": ["Read", "Write","List"],
      "credentials": [
        {
```

```
          "accessKey": "GHIJKL",
          "secretKey": "789012"
        }
      ]
    }
  ]
}
```

On lance l'ensemble des services : `docker compose up -d`

3. Programmation de l'api

L'api permet l'upload de fichier et la récupération d'une url directe pour accéder à l'image. L'api est développée en php/slim et, dans cette api, le service en charge de la communication avec le service S3 est programmé en utilisant le SDK AWS.

on a donc les dépendances suivantes :

```
{
  "require": {
    "php": "^8.2",
    "slim/slim": "^4.0",
    "slim/psr7": "^1.0",
    "php-di/php-di": "^7.0",
    "aws/aws-sdk-php": "^3.0",
    "vlucas/phpdotenv": "^5.6.0"
  }
}
```

La documentation complète du sdk php AWS est disponible ici : https://docs.aws.amazon.com/sdk-for-php/v3/developer-guide/welcome.html

3.1. Création du client s3

La manipulation de l'api S3 avec le SDK nécessite la création d'un client, instance de `Aws\S3\S3Client`. La documentation complète des opérations proposées par l'api de ce client est disponible ici : https://docs.aws.amazon.com/aws-sdk-php/v3/api/api-s3-2006-03-01.html

La création de ce client nécessite quelques paramètres :

- les credentials, i.e. accessKey + secretKey
- le endpoint : url d'accès au service S3, par exemple dans la configuration indiquée dans le docker-compose.yml ci-dessus : `http://S3:8333`
- une configuration obligatoire dès qu'on utilise un stockage compatible S3 et non AWS S3 : `'use_path_style_endpoint' ⇒ true`
- le paramètre `region` présent pour des raisons de compatibilité AWS : il s'agit de la region du

cloud AWS que l'on souhaite utiliser ; dans notre cas on peut utiliser une valeur quelconque.

La création du client est gérée par le conteneur d'injection de dépendances. On utilise donc les définitions PHP-DI ci-dessous. Par souci de sécurité et d'évolutivité, les valeurs de configuration sont stockées dans des variables d'environnement :

```
use Aws\S3\S3Client;
use Psr\Container\ContainerInterface;

return [
    'endpoint' => $_ENV['S3_ENDPOINT'],
    'region' => $_ENV['S3_REGION'] ?? 'seaweedFS',
    'key' => $_ENV['S3_ACCESS_KEY'],
    'secret' => $_ENV['S3_SECRET_KEY'],
    'bucket' => $_ENV['S3_BUCKET'],

    'S3_client'=> function(ContainerInterface $c){
        return new S3Client([
            'region'   => $c->get('region'),
            'endpoint' => $c->get('endpoint'),
            'use_path_style_endpoint' => true,
            'credentials' => [
                'key'    => $c->get('key'),
                'secret' => $c->get('secret'),
            ]
        ]);
    }
```

3.2. Le service de gestion du stockage (StorageService)

Dans l'application, le StorageService est le service métier en charge des opérations d'interaction avec le stockage S3 géré par SeaweedFS. Il s'appuie sur le sdk aws pour réaliser ces opérations. Dans notre application, il réalise deux fonctionnalités :

1. le stockage d'un fichier contenant une image dans un objet dont il génére la clé,
2. l'obtention d'une URL d'accès directe à l'objet, qui sera utilisée dans le frontend de l'application pour afficher l'image.

Le client S3 est injecté dans le service au travers de son constructeur. Le service reçoit également le nom du bucket à utiliser.

```
use AWS\S3\S3Client;
use AWS\S3\Exception\S3Exception;
use Psr\Http\Message\StreamInterface;

class StorageService
{
    private S3Client $s3_client;
```

```
    private string $bucket;

    public function __construct(S3Client $client, string $bucket) {
        $this->s3_client = $client;
        $this->bucket = $bucket;
    }
}
```

3.2.1. stockage d'une image

Pour stocker un contenu dans un bucket, on utilise la méthode du sdk `putObject()`. Cette méthode reçoit un tableau contenant la description de l'objet à créer :

- le nom du bucket à utiliser - REQUIRED - si le bucket n'existe pas, il est créé,
- la cké à utiliser - REQUIRED,
- le contenu, sous la forme d'un stream, d'une string, ou d'un descripteur de fichier,
- le type mime
- un droit d'accès, par défaut 'private'
- des métadonnées éventuelles

Le choix qui est fait pour les clés et le format suivant :

```
users/{user_id}/{uuid}.ext
users/michel_platoche/63559acdab8ed44a25e698776b77a758.png
```

Le contenu du fichier est reçu sous la forme d'un stream. La méthode `store()` du `StorageService` réalise 3 étapes :

1. elle génère la clé sous la forme choisie, en incluant un uuid,
2. elle stocke le fichier dans le stockage S3, en ajoutant le Content-Type et la date,
3. elle retourne la clé en résultat

```
public function store(
        string $id,
        StreamInterface $content,
        string $mime_type
    ): string {
        // génération de la clé
        $uuid = $this->generateUuid();
        $ext = $this->mimeToExtension($mime_type);
        $key = sprintf('users/%s/%s.%s', $id, $uuid, $ext );

       try {
            // stockage dans le bucket : on utilise le client interne
            $this->s3_client->putObject([
```

```
                'Bucket' => $this->bucket,
                'Key' => $key,
                'Body' => $content,
                'ContentType' => $mime_type,
                'Metadata' => [
                    'date' => date('d/m/Y H:i:s'),
                ]
            ]);
        } catch (S3Exception $e) {
            throw new StorageServiceException('erreur S3:'.$e->getMessage(), $e);
        }
        return $key;
    }
```

L

3.2.2. génération d'une URL directe pré-signée

La génération d'une url pré-signée va permettre de fournir au frontend de l'application une URL d'accès direct à un objet stocké dans un bucket, bien que ce bucket soit privé.

Cette url permet à son détenteur un accès temporaire à l'objet concerné. Il faut donc fournir une date d'expiration lors de la création de l'URL.

De plus, cette URL est créé pour une *opération particulière*. Dans notre cas, on va générer des URL pour accéder en lecture à un objet (`'GetObject'`), mais il est possible de générer des URL pour d'autres opérations, par exemple `'PutObject'`.

La génération de l'URL pré-signée se fait en deux étapes :

1. création de la commande pour laquelle on génère l'URL
2. génération de l'URL pour cette commande, avec une date d'expiration

```
public function getPresignedUrl(
        string $key,
        int $expiresInSeconds = 3600
    ): string {
        // création de la commande d'accès à l'objet
        $command = $this->s3_client->getCommand('GetObject', [
            'Bucket' => $this->bucket,
            'Key'    => $key,
        ]);
        // génération de l'URL pré-signée
        return (string) $this->s3_client
            ->createPresignedRequest($command, "+{$expiresInSeconds} seconds")
            ->getUri();
    }
```

Le résultat est une URL ayant cette forme :

```
# clé s3 :
users/michel_platoche/63559acdab8ed44a25e698776b77a758.jpg

# pre-signed url :
http://localhost:8333/demo_bucket/users/michel_platoche/63559acdab8ed44a25e698776b77a7
58.jpg?X-Amz-Content-Sha256=UNSIGNED-PAYLOAD&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-
Credential=ABCDEF%2F20260319%2FSeaweedFS%2Fs3%2Faws4_request&X-Amz-
Date=20260319T082804Z&X-Amz-SignedHeaders=host&X-Amz-Expires=3600&X-Amz-
Signature=75896d175c997cdc4422b223132227fca8fa23628035816729a2cf7968f95052
```

Cette url permet un accès direct à l'image depuis, par exemple, un navigateur. La réponse à une requête de type GET sur cette URL peut-être :

- `200 OK` + le contenu de l'objet + un `Content-Type` avec le type mime de l'objet,
- `400 BadRequest` si l'url est malformée ou a été modifiée,
- `403 Forbidden` si l'url a expirée

3.2.3. Cas particulier de la génération d'URL avec un stockage dockerisé au sein d'une composition

L'URL pré-signée est générée par le client S3 sur la base de la valeur du endpoint qui lui a été fournie lors de sa création. On se heurte à un problème classique dans le cas d'un environnement docker. Le souci est que l'URL pré-ignée est générée avec le hostname interne Docker (dans notre cas: S3:8333), mais le client externe qui utilisera l'url a besoin d'un hostname public (ex: localhost:8333).

Il y a plusieurs approches pour résoudre ce problème. L'approche recommandée qui est aussi la plus simple consiste à configurer deux endpoint et instancier deux clients S3 :

- un client instancié avec le endpoint interne (i.e. le service docker S3 au sein de la composition), pour les opérations (putObject, delete…)
- un client instancié avec le endpoint externe pour la génération des urls pré-signées.

Dans notre application Dockerisé, on utilise en fait :

- un endpoint S3 interne utilisé à l'interieur de la composition docker : `http://S3:8333`
- un endpoint S3 externe, utilisé pour les URL directs : `http://localhost:8333`
- deux clients S3 injectés dans le StorageService, l'un configuré avec le endpoint interne et utilisé pour les opérations internes, et l'autre configuré avec le endpoint externe et utilisé pour générer les URL directes :

```php
class StorageService
{
    private S3Client $s3_internal_client;
    private S3Client $s3_external_client;
    private string $bucket;
```

```
    public function __construct(S3Client $client, S3Client $s3_external_client, string
$bucket) {
        $this->s3_internal_client = $client;
        $this->s3_external_client = $s3_external_client;
        $this->bucket = $bucket;
    }
}
```

On doit donc créer deux client S3 dans les définitions PHP-DI et les injecter lors de la création du StorageService.

```
return [
     // interne à la composition docker, e.g. http://S3.service:8333
    'internal_endpoint' => $_ENV['S3_INTERNAL_ENDPOINT'],
    // externe, e.g. http://localhost:8333 /
    'external_endpoint' => $_ENV['S3_EXTERNAL_ENDPOINT'],
    'region' => $_ENV['S3_REGION'] ?? 'seaweedFS',
    'key' => $_ENV['S3_ACCESS_KEY'],
    'secret' => $_ENV['S3_SECRET_KEY'],
    'bucket' => $_ENV['S3_BUCKET'],

    'S3_internal_client'=> function(ContainerInterface $c){
        return new S3Client([
            'region'   => $c->get('region'),
            'endpoint' => $c->get('internal_endpoint'),
            'use_path_style_endpoint' => true,
            'credentials' => [
                'key'    => $c->get('key'),
                'secret' => $c->get('secret'),
            ],
        ]);
    },
    'S3_external_client'=> function(ContainerInterface $c){
        return new S3Client([
            'region'   => $c->get('region'),
            'endpoint' => $c->get('external_endpoint'),
            'use_path_style_endpoint' => true,
            'credentials' => [
                'key'    => $c->get('key'),
                'secret' => $c->get('secret'),
            ],
        ]);
    },

    StorageService::class => function(ContainerInterface $c){
        return new StorageService($c->get('S3_internal_client'),
                                  $c->get('S3_external_client'),
                                  $c->get('bucket'));
    }
```

```
]
```

3.3. Programmation de l'api Rest

L'api REST de notre application est réalisé en Slim et expose une route permettant à un utilisateur d'uploader un fichier contenant une image.

```
app->post('/users/{id}/photos', \storage\api\actions\UploadAction::class);
```

L'action associée, `UploadAction` utilise le `StorageService` pour enregistrer l'image dans le stockage S3 et obtenir une URL pré-signée pour accéder à cet objet.

Cette action :

1. récupère les données de la requête,
2. implante le pipeline de validation de ces données,
3. appelle la méthode `store()` du `StorageService` pour stocker le fichier et récupérer sa clé,
4. appelle la méthode `getPresignedUrl()` du `StorageService` pour obtenir l'url directe,
5. construit et retourne une réponse avec un body json.

```
class UploadAction
{
    private StorageService $storageService;

    private const array ALLOWED_MIME = [
        'image/jpeg',
        'image/png',
        'image/webp',
        'image/gif',
    ];
    // Taille maximale : 10 Mo
    private const int MAX_SIZE = 10 * 1024 * 1024;

    public function __construct(StorageService $storageService) {
        $this->storageService = $storageService;
    }

    public function __invoke(
        ServerRequestInterface $request,
        ResponseInterface $response,
        array $args
    ): ResponseInterface {

        $id = $args['id'];
        $files = $request->getUploadedFiles();
```

```
        // 1. Fichier présent ?
        if (empty($files['photo'])) {
            throw new HttpBadRequestException($request, 'pas de fichier présent');
        }

        /** @var UploadedFileInterface $upload */
        $upload = $files['photo'];

        // 2. Erreur d'upload PHP ?
        if ($upload->getError() !== UPLOAD_ERR_OK) {
            throw new HttpBadRequestException($request, 'erreur upload : '. $upload-
>getError());
        }

        // 3. Type MIME autorisé ?
        $mimeType = $upload->getClientMediaType();
        if (!in_array($mimeType, self::ALLOWED_MIME, true)) {
            throw new HttpBadRequestException($request, 'erreur upload : type prohibé
: '. $mimeType);

        }

        // 4. Taille acceptable ?
        if ($upload->getSize() > self::MAX_SIZE) {
            throw new HttpBadRequestException($request, 'erreur upload : trop
volumineux' . $upload->getSize());
        }

        try {
            // stockage dans le storage service et calcul de l'url présignée
            $key = $this->storageService->store($id, $upload->getStream(), $mimeType);
            $url = $this->storageService->getPresignedUrl($key);
        } catch (StorageServiceException $e) {
            throw new HttpInternalServerErrorException($request, 'erreur stockage : '.
$e->getMessage());
        }

        $response->getBody()->write(json_encode(['key'=>$key, 'url'=>$url]));
        return $response->withStatus(201);

    }
}
```

4. Administration du stockage S3, CLI

Il peut être nécessaire d'effectuer des opérations d'administration directement dans le stockage S3. Ces opérations peuvent être programmées avec le sdk, mais il est n général plus rapide d'intervenir directement.

Il existe deux possibilités complémentaires :

- utiliser l'interface d'administration de SeaweedFS
- utiliser `aws s3`, le client CLI fourni par Amazon.

4.1. Utilisation de l'interface d'administration de SeaweedFS

Il s'agit d'une Admin UI intégrée à SeaweedFS, qui propose une interface web d'administration du serveur. Elle permet de nombreuses opérations, dont les opérations spécifiques à SeaweedFS. Cette interface est disponible sur le port `23646`.

L'interface doit être explicitement lancée. Il faut le faire à **l'intérieur du conteneur docker**. La commande `weed admin` lance l'interface sur le port précisé, dont la valeur par défaut est 23646. L'option `-masters` permet de préciser l'url du serveur à administrer. La valeur par défaut est localhost:9333. Il est possible d'administrer plusieurs masters. Enfin, les options `-adminUser` et `-adminPassword` permettent d'exiger une authentification pour l'accès à l'interface.

```
# weed admin -port=23646 -adminUser=marcel -adminPassword=123456
-masters=localhost:9333
Starting SeaweedFS Admin Interface on port 23646
```

4.2. Utilisation du client cli aws s3

L'outil AWS CLI est un client cli permettant d'interagir en ligne de commande avec l'ensemble des services AWS, et donc en particulier avec un stockage S3 quelconque.

Vous devez d'abord installer ce client, sur votre **machine hôte**. Les instructions pour les différents OS sont disponibles ici : https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

Vous devez ensuite configurer votre client de façon à ce qu'il utilise votre installation locale, et non pas AWS. L'opération va conduire à créer un profil local stocké dans un fichier sur votre hôte. Dans un terminal de votre machine hôte :

```
aws configure --profile local
# Access Key ID: ABCDEF
# Secret Access Key: 123456
# Default Region: SeaweedFS
# Output format: json
```

L'opération crée le fichier `~/.aws/config` qu'il faut ensuite ouvrir et compléter en y ajoutant le endpoint S3. **Attention**, il faut bien sûr utiliser le endpoint externe puisque on utilise le client depuis l'hôte.

```
[profile local]
region = SeaweedFS
output = json
endpoint_url = http://localhost:8333
```

Vous pourrez ensuite exécuter des commandes en ligne pour manipuler votre stockage S3. Une description complète des commandes disponibles est disponible ici : https://docs.aws.amazon.com/cli/latest/userguide/cli-services-s3-commands.html.

Pour toutes les commandes exécutées dans le terminal, il faudra utiliser l'option `--profile local` de façon à utiliser le profil configuré.

Quelques exemples de commandes :

```
# lister les buckets

% aws s3 ls --profile local
2026-03-17 12:00:02 photos
```

```
# lister le contenu d'un bucket

% aws s3 ls s3://photos --profile local
        PRE /users
```

```
# uploader un fichier vers un bucket -
# attention, la clé utilisée est le nom du fichier

% aws s3 cp  agdujnsbjfjbxhqsdj5678.jpg s3://photos --profile local
upload: ./agdujnsbjfjbxhqsdj5678.jpg to s3://photos/agdujnsbjfjbxhqsdj5678.jpg
```

```
# télécharger un fichier

% aws s3 cp  s3://photos/agdujnsbjfjbxhqsdj5678.jpg  ./image.jpg --profile local
download: s3://photos/agdujnsbjfjbxhqsdj5678.jpg to ./image.jpg
```

```
# obtenir une url directe présignée

% aws s3 presign  s3://photos/agdujnsbjfjbxhqsdj5678.jpg   --profile local
http://localhost:8333/photos/agdujnsbjfjbxhqsdj5678.jpg?X-Amz-Algorithm=AWS4-HMAC-
SHA256&X-Amz-Credential=ABCDEF%2F20260319%2FSeaweedFS%2Fs3%2Faws4_request&X-Amz-
Date=20260319T162131Z&X-Amz-Expires=3600&X-Amz-SignedHeaders=host&X-Amz-
Signature=0e408cbb2c63c36f3505d48c39cf278c03f49a40efe48e6d72dc0ecb4da2818c
```

```
# créer un bucket

% aws s3 mb s3://img-store --profile local
make_bucket: img-store

% aws s3 ls --profile local
2026-03-19 17:24:57 img-store
2026-03-17 12:00:02 photos
```